PELATIHAN FLUTTER · MATERI LANJUTAN

Clean Architecture di Flutter

Flutter adalah framework UI. Maka aturan bisnis, seperti cara menghitung IPK, harus tinggal di luar widget.

UI vs business logic

SOLID

MVVM

Tiga pertanyaan hari ini

01

Logika apa yang boleh ada di widget?

UI logic vs business logic, lewat kasus KHS dan IPK mahasiswa.

02

Bagaimana menyusun kelasnya?

Lima prinsip SOLID sebagai pedoman memecah tanggung jawab.

03

Bagaimana merakitnya di Flutter?

Pola MVVM: View, ViewModel, Model, seperti di Flutter Learning Pathway.

STUDI KASUS

Halaman KHS di aplikasi kampus

Aplikasi mengambil nilai mata kuliah dari server, menghitung IPK, lalu menampilkan predikat kelulusan.

Aturannya milik kampus: bobot huruf A = 4, B = 3, C = 2, D = 1, E = 0, dan IPK ≥ 3,51 berpredikat Cumlaude.

KHS · Semester Ganjil

Mata kuliah	SKS	Nilai
Pemrograman Mobile	3	A
Basis Data	3	B
Kecerdasan Buatan	2	A

IPK 3,63 · Cumlaude

Sebelum: semua logika di dalam widget

```
@override
Widget build(BuildContext context) {
  final res = http.get(...); // ambil data
  double total = 0; int sks = 0;
  for (final n in nilai) {
    final bobot = n['huruf'] == 'A' ? 4
      : n['huruf'] == 'B' ? 3 : 2;
    total += bobot * n['sks'];
    sks += n['sks'] as int;
  }
  final ipk = total / sks;
  final predikat = ipk ≥ 3.51
    ? 'Cumlaude' : 'Memuaskan';
  return Text('IPK $ipk · $predikat');
}
```

Baris merah bukan urusan tampilan.

Pengambilan data, bobot nilai, dan aturan predikat ikut berjalan setiap kali layar di-*build* ulang.

Bug tersembunyi: bobot D dan E dianggap 2, dan pembagian nol bila belum ada nilai.

Mengapa itu jadi masalah

Sulit diuji

Untuk mengetes rumus IPK, kita harus menjalankan widget, emulator, dan server.

Tidak bisa dipakai ulang

Halaman transkrip dan halaman wisuda butuh rumus yang sama, lalu kodenya disalin.

Aturan berubah, UI ikut dibongkar

Kampus menambah nilai AB dan BC? Semua widget yang menghitung IPK harus diubah.

Boros dan lambat

build() bisa dipanggil puluhan kali; data diambil ulang dan dihitung ulang tiap kali.

Business logic: Dart murni, tanpa Flutter

```dart
// domain/kalkulator_ipk.dart
class NilaiMatkul {
  final String huruf; final int sks;
  const NilaiMatkul(this.huruf, this.sks);
}

class KalkulatorIpk {
  static const _bobot = {
    'A': 4, 'B': 3, 'C': 2, 'D': 1, 'E': 0};

  double hitung(List<NilaiMatkul> daftar) {
    final sks = daftar.fold(0, (a, n) ⇒ a + n.sks);
    if (sks == 0) return 0;
    final mutu = daftar.fold(0,
      (a, n) ⇒ a + _bobot[n.huruf]! * n.sks);
    return mutu / sks;
  }
}
```

```dart
// test/ipk_test.dart
test('IPK ganjil', () {
  final ipk =
    KalkulatorIpk().hitung([
    NilaiMatkul('A', 3),
    NilaiMatkul('B', 3),
    NilaiMatkul('A', 2)]);
  expect(ipk,
    closeTo(3.625, 0.001));
});
```

Diuji dalam milidetik dengan **flutter test**, tanpa emulator dan tanpa server.

UI logic: widget hanya menampilkan

```dart
// presentation/kartu_ipk.dart
class KartuIpk extends StatelessWidget {
  final double ipk;
  final String predikat;
  const KartuIpk({super.key,
    required this.ipk, required this.predikat});

  @override
  Widget build(BuildContext context) {
    final warna = predikat == 'Cumlaude'
      ? Colors.orange : Colors.blueGrey;
    return Card(child: ListTile(
      title: Text('IPK ${ipk.toStringAsFixed(2)}'),
      subtitle: Text(predikat,
        style: TextStyle(color: warna)),
    ));
  }
}
```

Baris biru tetap logika, tetapi logika tampilan: warna dan format dua desimal.

Widget menerima hasil jadi. Ia tidak tahu dari mana nilai datang atau bagaimana IPK dihitung.

Memilah logika di aplikasi akademik

Contoh	Jenis logika	Tempatnya
Menghitung IPK dari bobot huruf dan SKS	Business	Domain
Menentukan predikat Cumlaude (IPK ≥ 3,51)	Business	Domain
Syarat lulus: ≥ 144 SKS dan tanpa nilai E	Business	Domain
Mengambil KHS dari server SIAKAD	Akses data	Data (repository)
Menampilkan IPK dengan dua desimal	UI	Presentation
Warna oranye untuk predikat Cumlaude	UI	Presentation
Menampilkan loading dan tombol Coba lagi	UI	Presentation

Uji cepat: **apakah aturannya tetap sama bila aplikasi diganti menjadi web atau dicetak di kertas?** Jika ya, itu business logic.

Tiga lapisan clean architecture



> **Aturan ketergantungan: panah selalu menuju Domain.**
>
> Domain tidak mengimpor Flutter, http, atau database. Ganti server SIAKAD atau ganti UI, aturan IPK tidak tersentuh.

BAGIAN 2

SOLID: lima pedoman memecah kelas

S

Single Responsibility

Satu kelas, satu alasan untuk berubah.

O

Open / Closed

Terbuka untuk ditambah, tertutup untuk dibongkar.

L

Liskov Substitution

Turunan bisa menggantikan induknya tanpa kejutan.

I

Interface Segregation

Kontrak kecil sesuai kebutuhan pemakainya.

D

Dependency Inversion

Bergantung pada abstraksi, bukan implementasi.

S dan O pada kasus KHS

S · Single Responsibility

Pecah kelas yang punya tiga alasan berubah: server, aturan, tampilan.

```
// Salah: satu kelas, tiga tugas
class LayananKhs {
  Future<List> ambilDariServer() {...}
  double hitungIpk(List n) {...}
  Widget tampilkan() {...}
}

// Benar: satu kelas, satu tugas
class SiakadApi     { ambilKhs() }
class KalkulatorIpk { hitung() }
class KartuIpk      { build() }
```

O · Open / Closed

Program S2 punya batas Cumlaude 3,75? Tambah kelas baru, kode lama tak disentuh.

```
abstract class AturanPredikat {
  String predikat(double ipk);
}

class AturanS1 implements AturanPredikat {
  String predikat(double ipk) ⟹
    ipk ⩾ 3.51 ? 'Cumlaude' : 'Memuaskan';
}
class AturanS2 implements AturanPredikat {
  String predikat(double ipk) ⟹
    ipk ⩾ 3.75 ? 'Cumlaude' : 'Memuaskan';
}
```

L dan I pada kasus KHS

L · Liskov Substitution

Kode yang menghitung batas SKS semua mahasiswa tiba-tiba crash karena satu turunan.

```
class Mahasiswa {
  int maksSks(double ips) ⇒
    ips ⩾ 3 ? 24 : 18;
}

class MahasiswaCuti extends Mahasiswa {
  @override
  int maksSks(double ips) ⇒
    throw Exception('Cuti');
}
// Benar: kembalikan 0, janji induk
// (selalu ada angka) tetap ditepati
```

I · Interface Segregation

Aplikasi mahasiswa hanya membaca KHS. Jangan paksa ia mengenal fungsi milik dosen dan BAAK.

```
// Salah: satu kontrak gemuk
abstract class LayananAkademik {
  Future<List<NilaiMatkul>> ambilKhs();
  Future<void> ubahNilai();
  Future<void> cetakIjazah();
}
// Benar: kontrak kecil per pemakai
abstract class PembacaKhs {
  Future<List<NilaiMatkul>> ambilKhs();
}
abstract class PengubahNilai {
  Future<void> ubahNilai();
}
```

D · Dependency Inversion pada kasus KHS

```
// domain: kontrak (abstraksi)
abstract class NilaiRepository {
  Future<List<NilaiMatkul>> ambilKhs(String nim);
}

// data: implementasi nyata
class NilaiRepositorySiakad implements NilaiRepository {
  Future<List<NilaiMatkul>> ambilKhs(String nim) async {
    final res = await http.get(Uri.parse('$api/khs/$nim'));
    return parseNilai(res.body);
  }
}

// test: implementasi palsu, tanpa internet
class NilaiRepositoryPalsu implements NilaiRepository {
  Future<List<NilaiMatkul>> ambilKhs(String nim) async ⇒
    [NilaiMatkul('A', 3), NilaiMatkul('B', 3)];
}
```

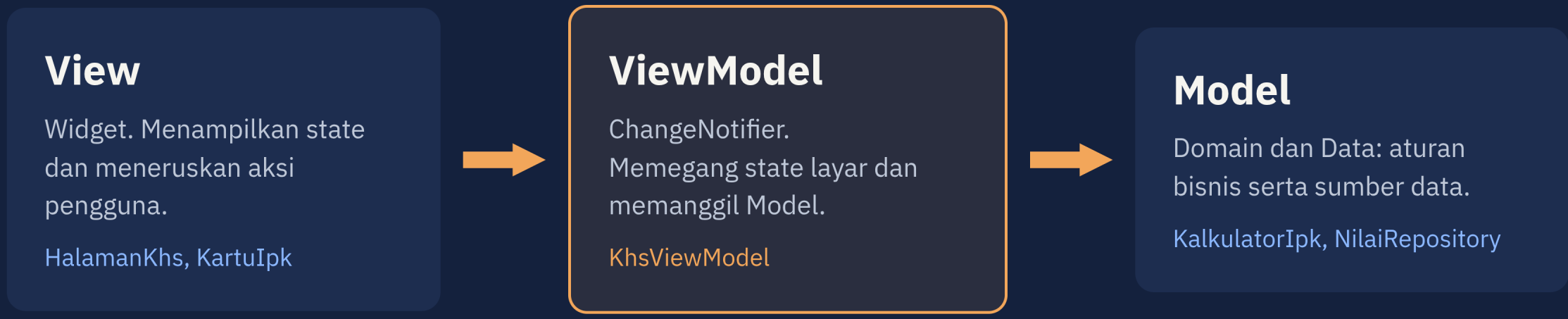
ViewModel menerima **NilaiRepository** lewat konstruktor, bukan membuat sendiri.

Aplikasi memberi versi SIAKAD; pengujian memberi versi palsu.

Inilah yang membuat panah dari lapisan Data menunjuk ke Domain.

BAGIAN 3

MVVM: merakit lapisan di Flutter



Arah balik tidak memakai panggilan langsung: ViewModel cukup memanggil **notifyListeners()**, lalu View yang mendengarkan membangun ulang dirinya.

ViewModel: state layar KHS

```
class KhsViewModel extends ChangeNotifier {
  KhsViewModel(this._repo, this._aturan);
  final NilaiRepository _repo;    // D: kontrak
  final AturanPredikat _aturan;   // O: bisa S1/S2
  final _kalkulator = KalkulatorIpk();

  bool memuat = false; String? error;
  double ipk = 0; String predikat = '';
  Future<void> muat(String nim) async {
    memuat = true; error = null; notifyListeners();
    try {
      final nilai = await _repo.ambilKhs(nim);
      ipk = _kalkulator.hitung(nilai);
      predikat = _aturan.predikat(ipk);
    } catch (_) { error = 'KHS gagal dimuat'; }
    memuat = false; notifyListeners();
  }
}
```

Tidak ada **import flutter/material** dan tidak ada widget.

Tiga keadaan layar: memuat, gagal, berhasil.

Rumus dan aturan didelegasikan ke Domain; ViewModel hanya mengatur alur.

View: mendengarkan, lalu menggambar

```dart
class HalamanKhs extends StatelessWidget {
  const HalamanKhs({super.key, required this.vm});
  final KhsViewModel vm;

  @override
  Widget build(BuildContext context) => ListenableBuilder(
    listenable: vm,
    builder: (context, _) {
      if (vm.memuat) return const CircularProgressIndicator();
      if (vm.error != null) return Text(vm.error!);
      return KartuIpk(ipk: vm.ipk, predikat: vm.predikat);
    },
  );
}

// main.dart: merakit semua bagian
final vm = KhsViewModel(NilaiRepositorySiakad(), AturanS1())
  ..muat('24.11.0001');
```

Mahasiswa S2? Ganti **AturanS1** menjadi **AturanS2**.

Uji tanpa internet? Ganti repository dengan versi palsu.

Dua perubahan itu tidak menyentuh satu baris pun di View.

Alur satu kali muat KHS



Setiap langkah tinggal di satu lapisan. Saat ada bug, kita tahu harus membuka file yang mana.

Yang dibawa pulang

Widget hanya berisi UI logic: format, warna, keadaan tampilan.

Aturan kampus tinggal di Domain sebagai Dart murni yang bisa diuji.

SOLID memandu cara memecah kelas; **MVVM** merakitnya di Flutter.

Checkpoint HOTS

C4 · Klasifikasikan logika di proyek akhir Anda ke View, ViewModel, Domain, atau Data.

C5 · Kapan memisah ke tiga lapisan justru berlebihan untuk aplikasi kecil?

C6 · Rancang kelas untuk aturan "maksimal SKS berdasarkan IPS" sesuai SOLID.